

# On the Repeating Group Finding Problem

Bo-Ren Kung, Wen-Hsien Chen, R.C.T Lee

Graduate Institute of Information Technology and Management  
Takming University of Science and Technology  
last123@gmail.com, wchen@im.ntu.edu.tw, rctlee@ncnu.edu.tw

## Abstract

In this paper, we investigate the repeating group detection problems. We define a special kind of repeating groups, namely maximal repeating group. Based upon this definition, we define a special string, called complete repeating group string. Using dynamic programming method, we can find all repeating groups of a string and determine whether a string is a complete repeating group string.

## 1 Introduction

String processing is an important and interesting task in computer science. It is also useful in DNA sequence associated research. In this task, there are some common and classic problems: local sequence alignment problem [1][2], global sequence alignment problem [3], multiple sequence alignment problem [4][5][6], exact pattern matching problem [7][8][9][10], approximate pattern matching problem [11][12][13], finding all maximal palindromes problem [14], finding all tandem repeats problem [14][15][16][17], finding all tandem arrays problem [18], etc. There are quite a lot of research done in these problems.

In this paper, we proposed two new and interesting problems for string processing

We shall first define some terminologies: A string is a sequence of characters. Thus we shall use  $T = t_1 t_2 \cdots t_n$  to denote a string. We shall use  $T(i, j)$  to denote  $t_i t_{i+1} \cdots t_j$ . In a string, if a substring  $T(i_1, j_1) = T(i_2, j_2)$  and  $i_2 > i_1$ , we say that  $T(i_1, j_1)$  and  $T(i_2, j_2)$  form a repeating group.

We further define a term, maximal repeating

group. A maximal repeating group of a string is a repeating group which is not contained in any other repeating group. For instance, consider  $T = cabcdabce$ . Then  $abc$  is a maximal repeating group while neither  $ab$  nor  $bc$  is a maximal repeating group because they are both contained in  $abc$ .

If  $T = A_1 A_2 \cdots A_m$  such that for every  $A_i$ , there is an  $A_j$  such that  $A_i$  and  $A_j$  form a maximal repeating group and no two  $A_j$  correspond to one  $A_i$ ,  $T$  is called a complete repeating group string.

Suppose the string is  $T = accaabbcc aacbcab$ . In this case  $T = A_1 A_2 A_3 A_4 A_5 A_6 A_7 A_8$  where  $A_1 = ac, A_2 = ca, A_3 = ab, A_4 = bc, A_5 = ca, A_6 = ac, A_7 = bc, A_8 = ab$ .

In other words, we can see that  $T = A_1 A_2 A_3 A_4 A_2 A_1 A_4 A_3$ . Note that all repeating groups in a complete repeating group string must be maximal and non-overlapping.

In this paper, we discuss two problems: Problem 1: Given a string  $T = t_1 t_2 \cdots t_n$ , find all repeating groups in  $T$ . Problem 2: Given a string  $T = t_1 t_2 \cdots t_n$ , decompose  $T$  into a complete repeating group string if possible.

## 2 Problem 1

In this section, we propose an algorithm to solve Problem 1. The algorithm is based on dynamic programming approach.

In Problem 1, we are given a string  $T = t_1 t_2 \cdots t_n$  and we have to find repeating groups of  $T$ . We now compare  $T(1, i)$  with  $T(1, j)$ . Without losing generality, we assume that  $i < j$ . Let  $M(i, j)$  denote the length of

the longest common suffix of  $T(1, i)$  and  $T(1, j)$ . The table containing all  $M(i, j)$ 's will be called the  $M$  table of  $T$ . For example, let  $T = gababagaba$ . Then we can see that  $M(3, 5) = 2$  because the longest common suffix of  $T(1, 5) = gabab$  and  $T(1, 3) = gab$  is  $ab$ . On the other hand,  $M(4, 7) = 0$  as there is no common suffix between  $T(1, 7) = gababag$  and  $T(1, 4) = gaba$ .

To create  $M$  table, we use the following recursive formula:

$$M(i, j) = \begin{cases} M(i-1, j-1) + 1 & \text{if } t_i = t_j \\ 0 & \text{if } t_i \neq t_j \end{cases}$$

Formula 1

The following dynamic programming table, Table 2-1, gives all  $M(i, j)$ 's of  $T = gababagaba$ .

| $i$ | $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|-----|-----|---|---|---|---|---|---|---|---|---|----|
| $j$ | $i$ | g | a | b | a | b | a | g | a | b | a  |
| 1   | g   |   |   |   |   |   |   |   |   |   |    |
| 2   | a   | 0 |   |   |   |   |   |   |   |   |    |
| 3   | b   | 0 | 0 |   |   |   |   |   |   |   |    |
| 4   | a   | 0 | 1 | 0 |   |   |   |   |   |   |    |
| 5   | b   | 0 | 0 | 2 | 0 |   |   |   |   |   |    |
| 6   | a   | 0 | 1 | 0 | 3 | 0 |   |   |   |   |    |
| 7   | g   | 1 | 0 | 0 | 0 | 0 | 0 |   |   |   |    |
| 8   | a   | 0 | 2 | 0 | 1 | 0 | 1 | 0 |   |   |    |
| 9   | b   | 0 | 0 | 3 | 0 | 2 | 0 | 0 | 0 |   |    |
| 10  | a   | 0 | 1 | 0 | 4 | 0 | 3 | 0 | 1 | 0 |    |

 Table 2-1 The  $M$  table of  $T = gababagaba$ .

From the above table, we can see that  $M(4, 6) = 3$  which denotes  $T(1, 4) = gaba$  and  $T(1, 6) = gababa$  have the longest common suffix  $aba$  with length 3. This means that we have found a repeating group  $(T(2, 4), T(4, 6))(aba)$ . Since  $M(4, 10) = 4$ , we have found another repeating group

$(T(1, 4), T(7, 10))(gaba)$ .

By examining all of  $T(i, j)$ 's, we will be able to find all of the repeating groups.

### 3 The Detection of Overlapping Repeating Groups and the Modification of the $M$ Table

We first define overlapping as follows: A substring  $T_1 = t_i t_{i+1} \cdots t_j$  overlaps with a substring  $T_2 = t_{i'} t_{i'+1} \cdots t_{j'}$  if a suffix of  $T_1$  is a prefix of  $T_2$ . For example, let  $T = ababa$ ,  $T_1 = T_{(1,3)} = aba$ ,  $T_2 = T_{(3,5)} = aba$ . We can easily see that the suffix of  $T_1$  is equal to the prefix of  $T_2$ . So we can call  $T_1$  and  $T_2$  an overlapping repeating group. For Problem 2, we do not allow overlapping repeating groups.

Suppose  $T = abababa$ . We may first get the  $M$  table as follows:

| $i$ | $j$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|-----|-----|---|---|---|---|---|---|---|
| $j$ | $i$ | a | b | a | b | a | b | a |
| 1   | a   |   |   |   |   |   |   |   |
| 2   | b   | 0 |   |   |   |   |   |   |
| 3   | a   | 1 | 0 |   |   |   |   |   |
| 4   | b   | 0 | 2 | 0 |   |   |   |   |
| 5   | a   | 1 | 0 | 3 | 0 |   |   |   |
| 6   | b   | 0 | 2 | 0 | 4 | 0 |   |   |
| 7   | a   | 1 | 0 | 3 | 0 | 5 | 0 |   |

 Table 3-1 The  $M$  table of  $T = abababa$ 

From the above table, we can see that  $M(5, 7) = 5$  which indicates that  $T(5-5+1, 5) = T(1, 5) = ababa$  and  $T(7-5+1, 7) = T(3, 7) = ababa$  is a repeating group. We can also see that  $T(3, 7)$  overlaps  $T(1, 5)$  with a common substring  $T(3, 5) = aba$ . Thus we shall not count  $T(1, 5) = ababa$  and  $T(3, 7) = ababa$  as a repeating group for Problem 2. In general, if  $M(i, j) = k$ , then  $T(i-k+1, i) = T(j-k+1, j)$ , or equivalently,  $T(i-k+1, i)$

and  $T(j-k+1, j)$  is a repeating group. If  $j-k+1 < i$ , this repeating group is overlapping as shown in Fig.1.

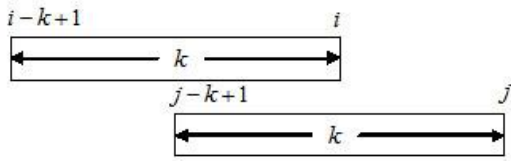


Fig 1 An Overlapping Repeating Group.

If  $M(i, j) = k$  and  $j-k+1 < i$ , we know that  $T(i-k+1, i) = T(j-k+1, j)$ . Thus  $T(i-k+1, i)$  and  $T(j-k+1, j)$  is an overlapping repeating group. But, as shown in Fig 2,  $T(j-k+1, i)$  and  $T(i+1, j)$  is still a non-overlapping repeating group.

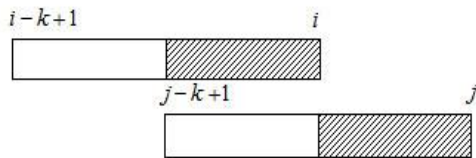


Fig 2 Non-overlapping Repeating Group of an Overlapping Repeating Group

In other words, we can use the following recursive formula to find the overlapping repeating groups and modify them if possible:

$$M(i, j) = j - i \text{ if } j - k + 1 < i$$

Formula 2

According to the above formula, we can transform Table 3-1 to Table 4-2.

| $i$ | $j$ | 1   | 2   | 3   | 4   | 5   | 6   | 7   |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
|     |     | $a$ | $b$ | $a$ | $b$ | $a$ | $b$ | $a$ |
| 1   | $a$ |     |     |     |     |     |     |     |
| 2   | $b$ | 0   |     |     |     |     |     |     |
| 3   | $a$ | 1   | 0   |     |     |     |     |     |
| 4   | $b$ | 0   | 2   | 0   |     |     |     |     |

|   |     |   |   |   |   |   |   |  |
|---|-----|---|---|---|---|---|---|--|
| 5 | $a$ | 1 | 0 | 2 | 0 |   |   |  |
| 6 | $b$ | 0 | 2 | 0 | 2 | 0 |   |  |
| 7 | $a$ | 1 | 0 | 3 | 0 | 2 | 0 |  |

Table 4-2 The  $M'$  table of  $T = abababa$

## 5 Problem 2

We are now ready to solve Problem 2. We first illustrate the general approach of our algorithm. Given a string  $T$ , we first find the longest suffix  $Y$  of  $T$  which is equal to a substring  $X$  of  $T$  which does not overlap with  $Y$  as shown in Fig 3.

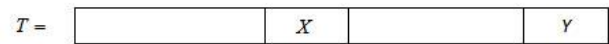


Fig 3  $X$  of  $T$  which does not overlap with  $Y$ .

If there is no such non-overlapping repeating group, report failure; otherwise let  $T' = T - X - Y$  as shown in Fig 4 and start the above process again.

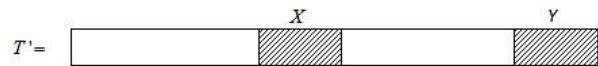


Fig 4 Non-overlapping repeating group.

Let us give an example here to illustrate our approach. Suppose we are given the following string:

1 2 3 4 5 6 7 8 9 10 11 12  
 $T = A \ b \ c \ d \ a \ c \ c \ d \ a \ c \ a \ b$

Using the algorithm given in the previous sections, we may immediately find the longest suffix  $ab$ , namely  $T(11,12)$ , which is equal to a substring in  $T$ , not overlapping with it, which is  $T(1,2)$ . We record this first pair  $T(1,2) = T(11,12) = ab$ .

We may now mark  $T(1,2)$  and  $T(11,12)$  and consider the remaining unmasked string as

follows:

1 2 3 4 5 6 7 8 9 10 11 12  
 $T = \begin{array}{cccccccccccc} \text{a} & \text{b} & \text{c} & \text{d} & \text{a} & \text{c} & \text{c} & \text{d} & \text{a} & \text{c} & \text{a} & \text{b} \end{array}$

We find that  $T(7,10) = T(3,6) = cdac$ .  
 Thus we conclude that  $T$  is a complete repeating group string.

Previously, we showed that we could use dynamic programming method to create a  $M$  table. We also showed that we can use Formula 2 to modify the table. Suppose that we have the following string:

$T = a b a b c a b c a b$

Then the  $M$  table looks like the following:

| $i$ |   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|-----|---|---|---|---|---|---|---|---|---|---|----|
| $j$ |   | a | b | a | b | c | a | b | c | a | b  |
| 1   | a |   |   |   |   |   |   |   |   |   |    |
| 2   | b | 0 |   |   |   |   |   |   |   |   |    |
| 3   | a | 1 | 0 |   |   |   |   |   |   |   |    |
| 4   | b | 0 | 2 | 0 |   |   |   |   |   |   |    |
| 5   | c | 0 | 0 | 0 | 0 |   |   |   |   |   |    |
| 6   | a | 1 | 0 | 1 | 0 | 0 |   |   |   |   |    |
| 7   | b | 0 | 2 | 0 | 2 | 0 | 0 |   |   |   |    |
| 8   | c | 0 | 0 | 0 | 0 | 3 | 0 | 0 |   |   |    |
| 9   | a | 1 | 0 | 1 | 0 | 0 | 4 | 0 | 0 |   |    |
| 10  | b | 0 | 2 | 0 | 2 | 0 | 0 | 5 | 0 | 0 |    |

Table 5-1 The  $M$  table of  
 $T = ababcabcb$

We use Formula 2 to modify the above table as follows:

| $i$ |   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|-----|---|---|---|---|---|---|---|---|---|---|----|
| $j$ |   | a | b | a | b | c | a | b | c | a | b  |
| 1   | a |   |   |   |   |   |   |   |   |   |    |

|    |   |   |   |   |   |   |   |   |   |   |  |
|----|---|---|---|---|---|---|---|---|---|---|--|
| 2  | b | 0 |   |   |   |   |   |   |   |   |  |
| 3  | a | 1 | 0 |   |   |   |   |   |   |   |  |
| 4  | b | 0 | 2 | 0 |   |   |   |   |   |   |  |
| 5  | c | 0 | 0 | 0 | 0 |   |   |   |   |   |  |
| 6  | a | 1 | 0 | 1 | 0 | 0 |   |   |   |   |  |
| 7  | b | 0 | 2 | 0 | 2 | 0 | 0 |   |   |   |  |
| 8  | c | 0 | 0 | 0 | 0 | 3 | 0 | 0 |   |   |  |
| 9  | a | 1 | 0 | 1 | 0 | 0 | 3 | 0 | 0 |   |  |
| 10 | b | 0 | 2 | 0 | 2 | 0 | 0 | 3 | 0 | 0 |  |

Table 5-2 The  $M'$  table of  
 $T = ababcabcb$

We now examine the last row of the above table and note the largest element in the row is  $M(7,10)$ . This means that we have found the longest suffix  $T(8,10)$  which is equal to another substring in  $T$  which is  $T(5,7)$ . This is our first non-overlapping repeating group found.

We mark these two substrings so that the table looks like the following:

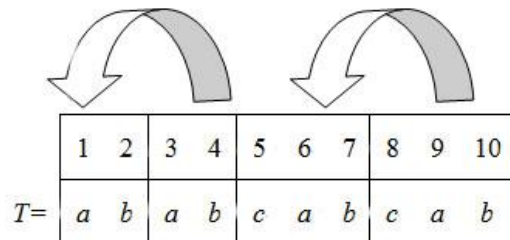
| $i$ |   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|-----|---|---|---|---|---|---|---|---|---|---|----|
| $j$ |   | a | b | a | b | c | a | b | c | a | b  |
| 1   | a |   |   |   |   |   |   |   |   |   |    |
| 2   | b | 0 |   |   |   |   |   |   |   |   |    |
| 3   | a | 1 | 0 |   |   |   |   |   |   |   |    |
| 4   | b | 0 | 2 | 0 |   |   |   |   |   |   |    |
| 5   | c | 0 | 0 | 0 | 0 |   |   |   |   |   |    |
| 6   | a | 1 | 0 | 1 | 0 | 0 |   |   |   |   |    |
| 7   | b | 0 | 2 | 0 | 2 | 0 | 0 |   |   |   |    |
| 8   | c | 0 | 0 | 0 | 0 | 3 | 0 | 0 |   |   |    |
| 9   | a | 1 | 0 | 1 | 0 | 0 | 3 | 0 | 0 |   |    |
| 10  | b | 0 | 2 | 0 | 2 | 0 | 0 | 3 | 0 | 0 |    |

Table 5-3 Marking the  $M$  table of  
 $T = ababcabcb$

For the remaining matrix, the last row is row 4. We find the largest element of this row is  $M(2,4)$ . This means that we have found

another non-overlapping repeating group, namely,  $T(3,4)$  and  $T(1,2)$ .

After marking the two substrings again, we find that the matrix becomes empty. We report our string looks like the following and is a complete repeating group string.



## 6 Conclusion

In this paper, we discuss two string processing problems. We show that the dynamic programming technique can be effectively used to solve these problems. In the future we would like to explore the following problem. Note that the decomposition of a string into complete repeating group string is not unique. It will be good if we can find solution such that the number of maximal repeating groups is minimized. We suspect that this problem may be NP-complete. If it is not, we hope to find a polynomial algorithm to solve the problem.

## References

- [1] Smith, T. F. and Waterman M. S.. Identification of Common Molecular Subsequences. 1982, pp. 195-197.
- [2] Webb, B. M., Liu, J. S., and Lawrence, C. E.. BALSA: Bayesian algorithm for local sequence alignment. Nucleic Acids Research, Vol. 30, No. 5, pp. 1268-1277.
- [3] Huang, X.. On global sequence alignment. Bioinformatics, Vol. 10, No. 3, 1994, pp. 227-335.
- [4] Carrillo, H. and Lipman, D. The Multiple Sequence Alignment Problem in Biology. SIAM Journal on Applied Mathematics, Vol. 48, No. 5, 1988, pp. 1073-1082.
- [5] Chan, S. C., Wong, A. K. C. and Chiu, D. K. Y.A .. Survey of Multiple Sequence Comparison Methods. Bulletin of Mathematical Biology, Vol. 54, No. 4, 1992, pp. 563-598.
- [6] Lipman, D. J., Altschul, S. F. and Kececioglu, J. D.. A Tool for Multiple Sequence Alignment. Proc. Nat. Acad. Sci., Vol. 86, 1989, pp. 4412-4415.
- [7] Aho, A. V. and Corasick, M. J.. Efficient String Matching, Matching. Communications of ACM, Vol. 18, 1975, pp. 333-340.
- [8] Boyer, R. S and Moore, J. S.. A Fast String Searching Algorithm. Communication of the ACM , Vol. 20 , 1977 , pp. 762-772 .
- [9] Fischer, M.M., and Paterson, M.S.. String-Matching and other products. SIAM-AMS Proceedings, Vol. 7., 1974, pp. 113-125.
- [10] Knuth, D. E., Morris, J. H. and Pratt, V. R.. Fast Pattern Matching in Strings. SIAM Journal on Computing , Vol. 6 , 1977 , pp. 323-350.
- [11] Landau, G. M. and Vishkin, U.. Efficient String Matching with k Mismatches. Theoretical Computer Science, Vol. 43, 1986, pp. 239-149.
- [12] Galil, Z. and Giancarlo, R. Improved String Matching with k Mismatches. SIGACT News, Vol. 17, No. 4, 1986, pp. 52-54.
- [13] Ukkonen E.. Algorithms for Approximate String Matching. Information and Control, Vol. 64, 1985, pp. 100-118.
- [14] Gusfield, D. Algorithms on Strings Trees and Sequences: Computer Science and Computational Biology. Cambridge University Press, 1997.
- [15] Benson, G. Tandem Repeats Finder: a Program to Analyze DNA Sequences. Oxford
- [16] Buchner, M. and Janjarsjitt, S.. Detection and Visualization of Tandem Repeats in DNA

- Sequences. IEEE Transactions on Signal Processing, Vol. 51, 2003, pp. 2280-2287.
- [17] Wexler, Y., Yakhini, Z., Kashi, Y. and Geiger, D. Finding Approximate Tandem Repeats in Genomic Sequences. Proceedings of the eighth annual international conference on Computational molecular biology, 2004, pp. 223-232.
- [18] Stoye, J., Gusfield, D.. Simple and flexible detection of contiguous repeats using a suffix tree. Theoretical Computer Science, 2002.